Before we start

- **Logistics**
  - Check in
  - Download the latest materials, especially class activities
- **Staying on track**
  - Mind the deadlines
  - Follow instructions for submissions
  - Don't fall behind: the course is short, and ramps up in complexity quickly
- **Preparation and support**
  - Come prepared: review the activity and slides beforehand
  - Get in touch with questions and concerns



Data Structures and Algorithms

Shortest Path Problem

Prof. Reza Skandari

Today

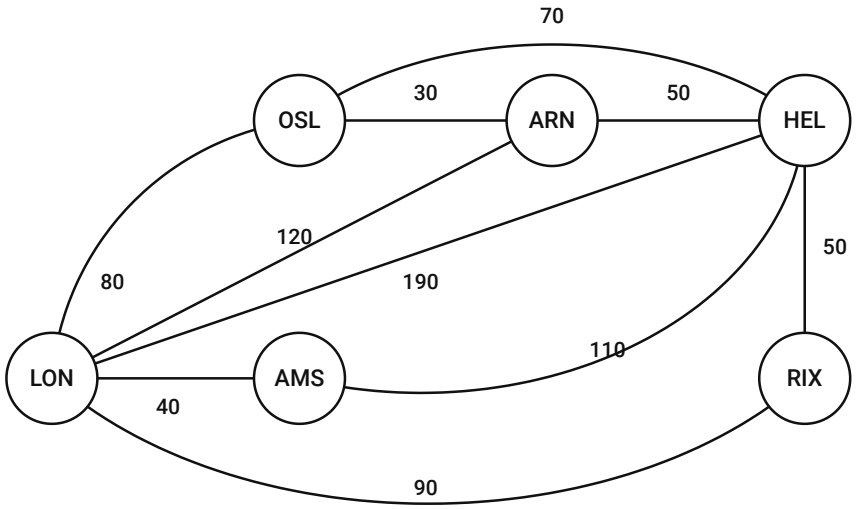
Dijkstra's algorithm for shortest paths:

1. Basic version: why it works
2. Efficient version: avoiding repeated work

Shortest paths

Suppose you're travelling around Europe

- Know flights and their prices
- Cheapest price?



What is the shortest path from LON to HEL using BFS?

Dijkstra's algorithm: basic version (sketch)

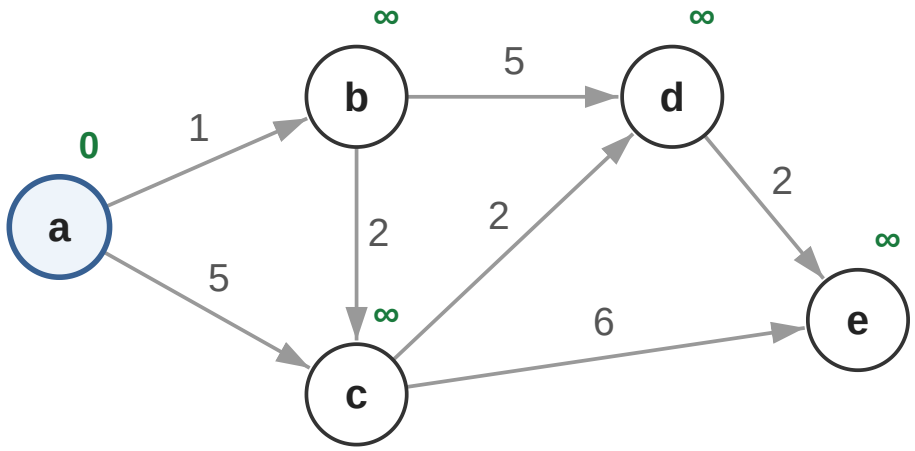
Input: graph G, start node s

Output: shortest distances (and paths) from s to all other nodes in G

1. **Initialisation**:
    - Mark s explored, set its distance to 0
    - Other nodes: unexplored, no distances
2. **Main loop**: while there are unexplored nodes:
    - Calculate "costs" of all edges from explored nodes to unexplored nodes
    - Pick the "cheapest" such edge
    - Mark its end node explored, and store its cost as its distance
3. (Reconstruct the shortest paths)

Stored distances will be shortest distances from s!

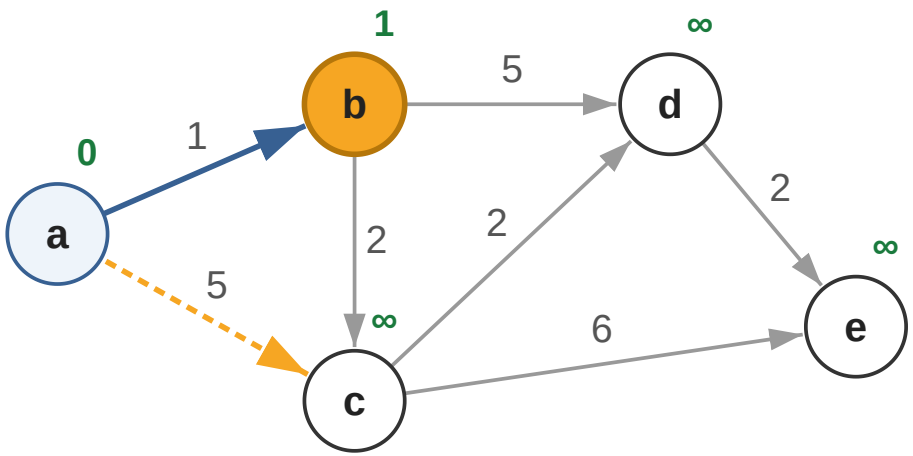
Dijkstra example: step 1 (initialise)



Mark `a` explored, distance 0.

Node	a	b	c	d	e
Distance	**0**	∞	∞	∞	∞

Dijkstra example: step 2

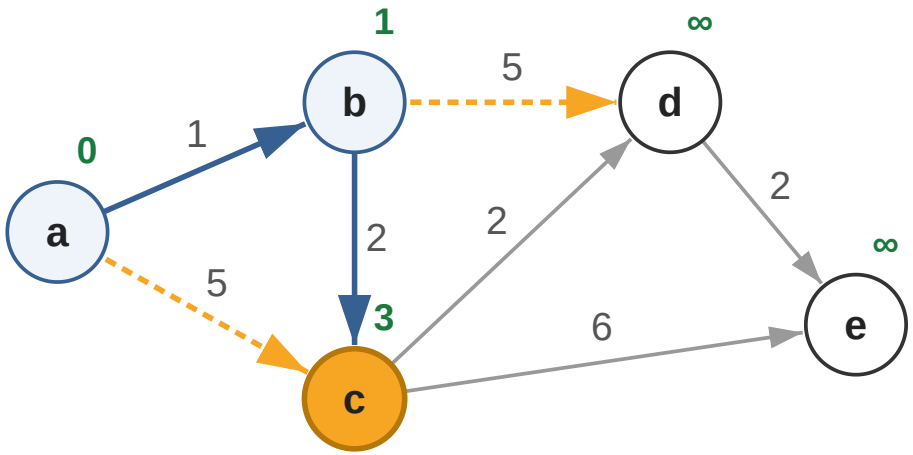


Candidate	Cost	Total
a → b	$0 + 1$	**1**
a → c	$0 + 5$	5

Explore `b`.

Node	a	b	c	d	e
Distance	0	**1**	∞	∞	∞

Dijkstra example: step 3

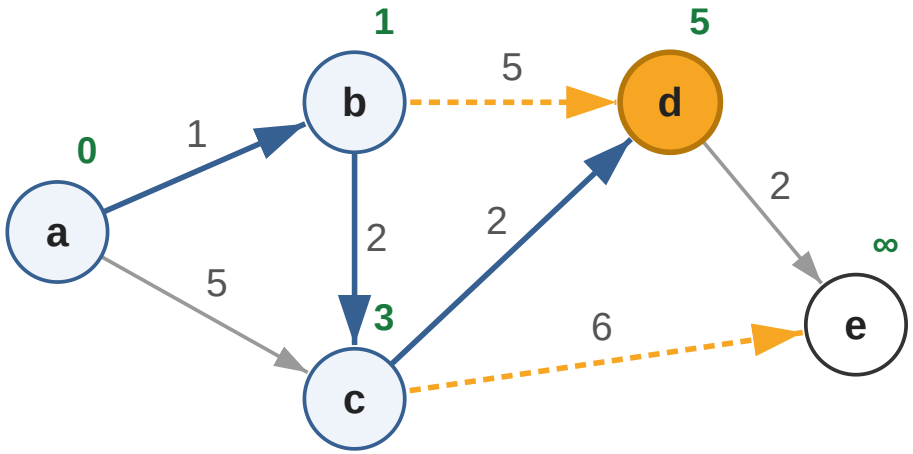


Candidate	Cost	Total
a → c	$0 + 5$	5
b → c	$1 + 2$	**3**
b → d	$1 + 5$	6

Explore `c`.

Node	a	b	c	d	e
Distance	0	1	**3**	∞	∞

Dijkstra example: step 4

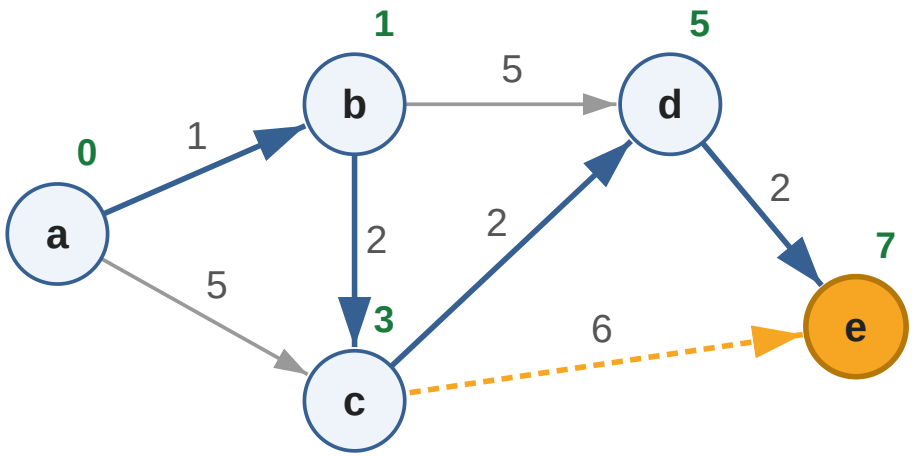


Candidate	Cost	Total
b → d	$1 + 5$	6
c → d	$3 + 2$	**5**
c → e	$3 + 6$	9

Explore `d`.

Node	a	b	c	d	e
Distance	0	1	3	**5**	∞

Dijkstra example: step 5



Candidate	Cost	Total
c → e	$3 + 6$	9
d → e	$5 + 2$	**7**

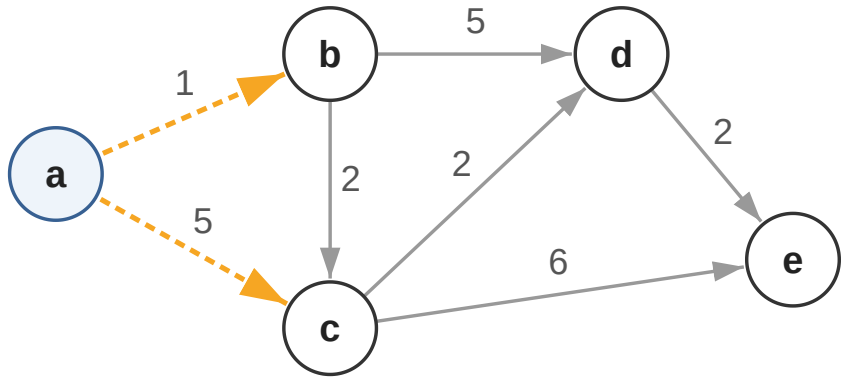
Explore `e`: all nodes explored, done!

Node	a	b	c	d	e
Distance	0	1	3	5	**7**

Why it works: first iteration

Claim: in the **first iteration**, we find the shortest distance to the node we explore

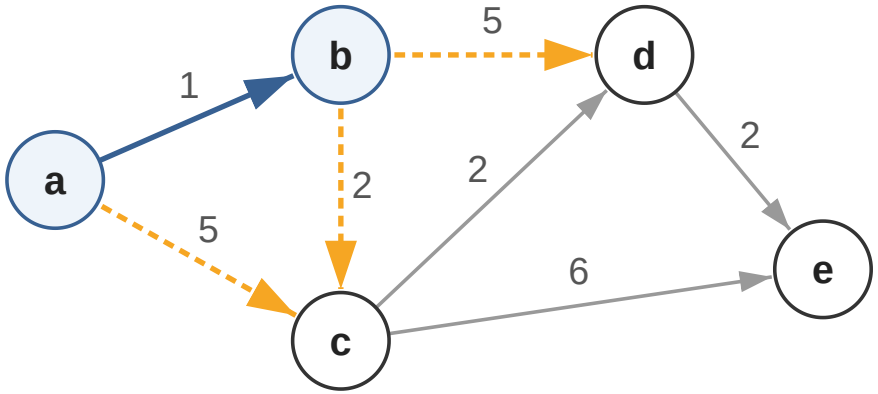
- Lowest-cost edge, explored → unexplored:
  - Must be the shortest distance to that node
  - Here `a` → `b` must give shortest distance to `b`:
    - Any other path would need to take a costlier route through `c`
  - We then know shortest distances to the two explored nodes



Why it works: second iteration

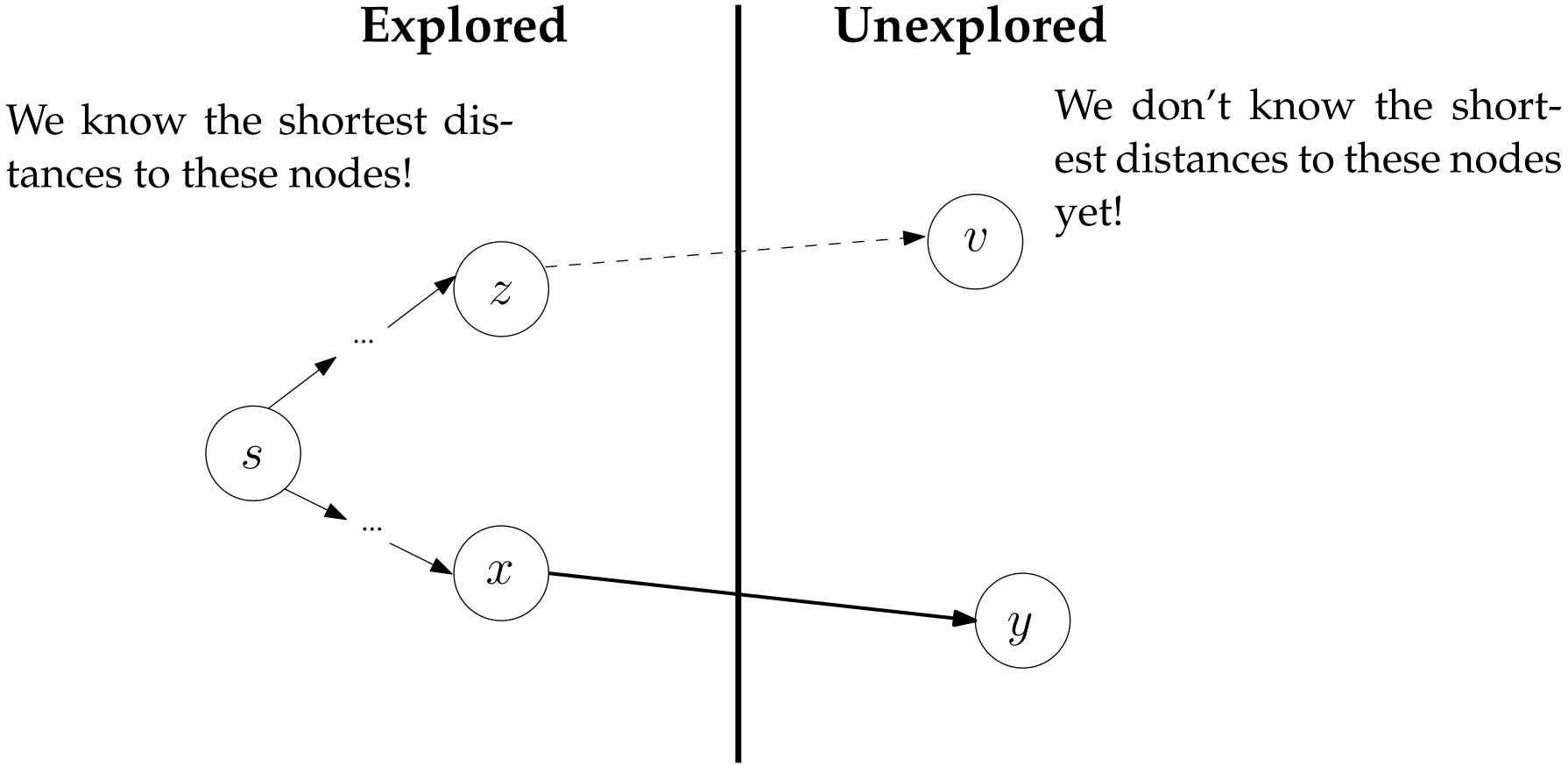
Claim: in the **second iteration**, we find the shortest distance to the node we explore

- Lowest-cost edge, explored → unexplored (`b` → `c`):
  - We already know shortest distances to `a`, `b`
  - Must give the shortest distance to `c`
  - A shorter path to `c` would need `a` → `c` or `b` → `d`
  - But `b` → `c` is lowest cost: 3 vs 5 and 6

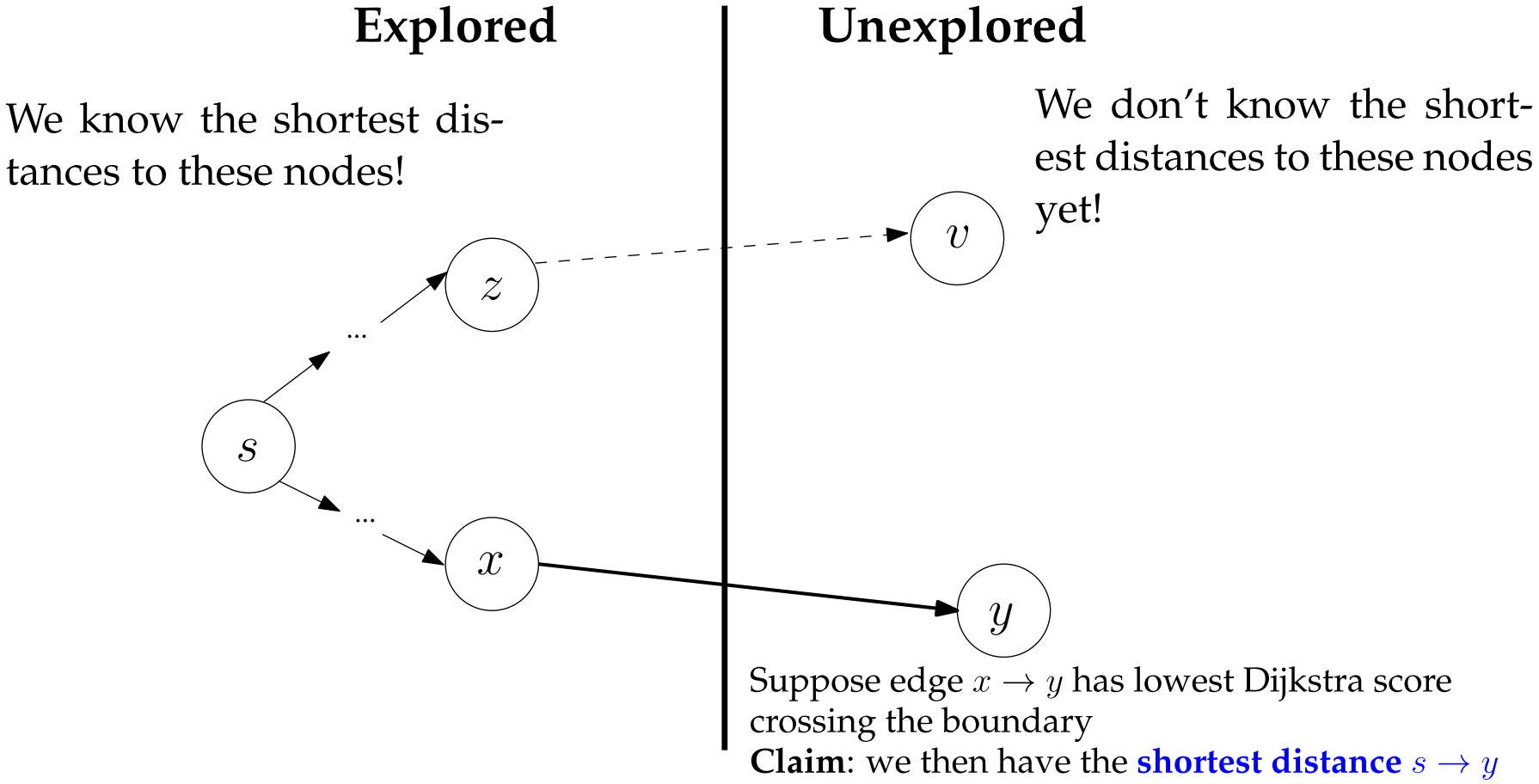


Why it works: any iteration

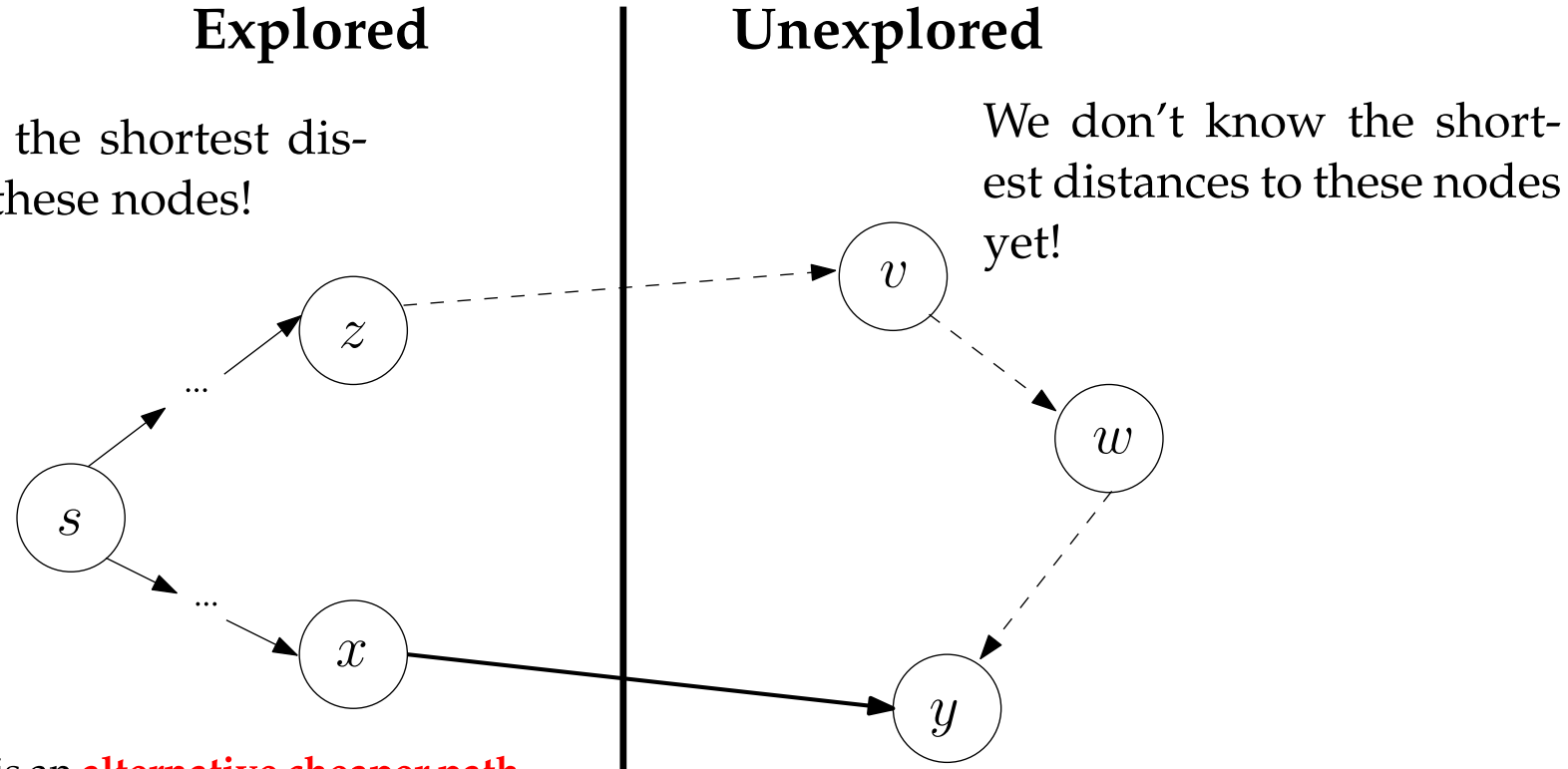
Claim: in **any iteration**, we find the shortest distance to the node we explore



Why it works: any iteration



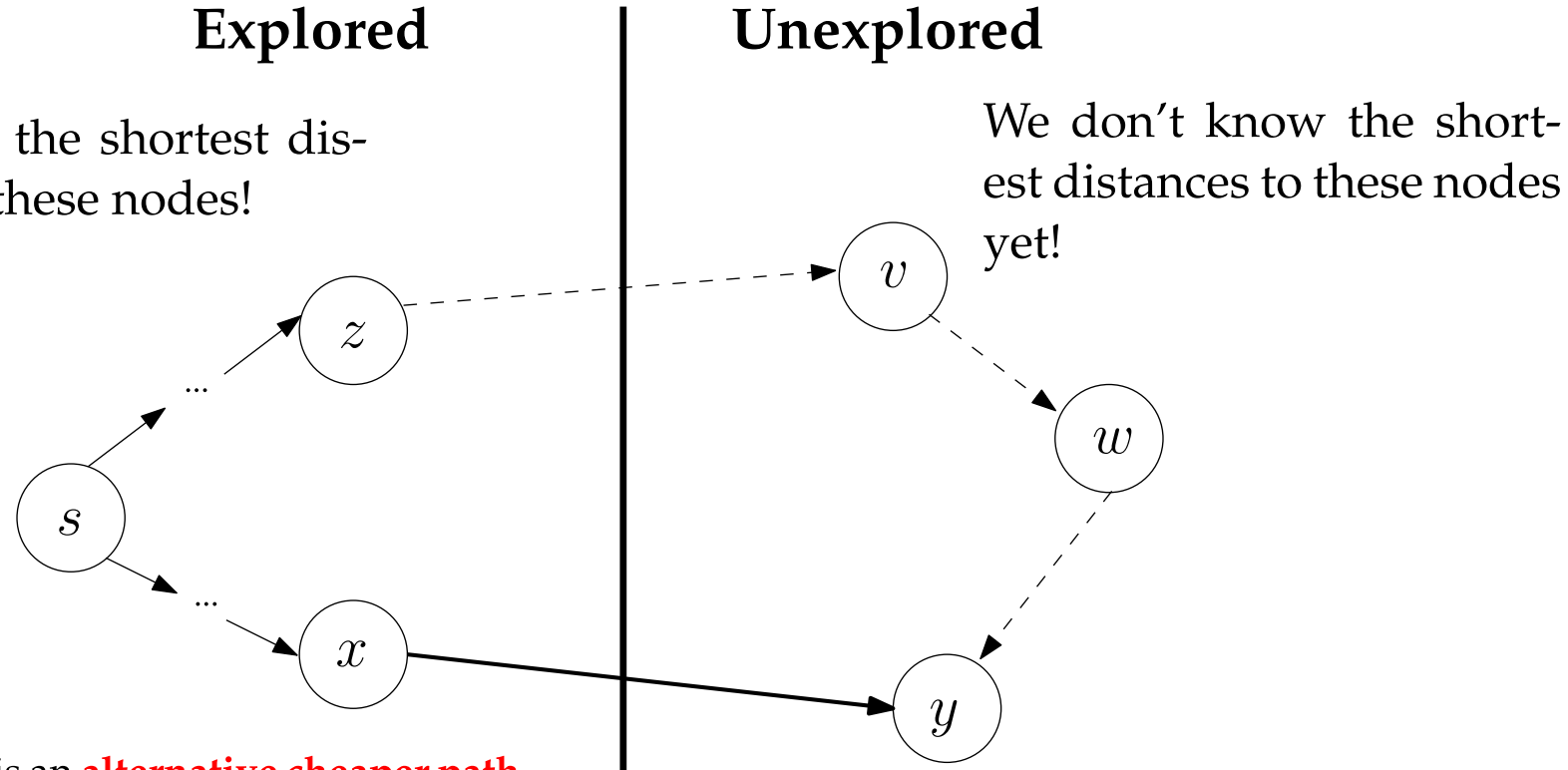
Why it works: any iteration



Suppose there is an **alternative cheaper path**
$s \to \ldots \to z \to v \to w \to \ldots \to y$
It also has to cross the boundary
from explored to unexplored

Suppose edge $x \to y$ has lowest Dijkstra score crossing the boundary
Claim: we then have the **shortest distance** $s \to y$

Why it works: any iteration



Suppose there is an **alternative cheaper path**
$s \to \ldots \to z \to v \to w \to \ldots \to y$
It also has to cross the boundary
from explored to unexplored

But $x \to y$ is the **cheapest** way to cross the boundary
(because the algorithm picked it!)
Therefore $s \to \ldots \to z \to v$ has higher cost than $s \to \ldots \to x \to y$

Suppose edge $x \to y$ has lowest Dijkstra score crossing the boundary
Claim: we then have the **shortest distance** $s \to y$

Contradiction!

So getting to y through the alternative route is more costly than through x
There is no cheaper path from s to y than through $x \to y$

Let's make it more precise

Input: (directed, connected) graph $G = (V, E)$

- V: set of n vertices, E: set of m edges
- Each edge e has length (cost) $c_e \geq 0$ (important!)
- Start from vertex s

Output: for each vertex v in V:

- length of shortest $s \to v$ path in G

Dijkstra's algorithm

For a graph $G = (V, E)$: V: n vertices, E: m edges, edge (v, w) has length c_{vw}

Initialise: starting vertex s

- Set of vertices gone through so far $X = \{s\}$ (not gone through $V - X$)
- Shortest distances to all vertices A: $A[s] = 0$

Main loop: While $X \neq V$: (not all nodes explored)

- Go through all edges (v, w) starting in X and ending in $V - X$
- Pick the edge that minimizes $A[v] + c_{vw}$, call it (v^*, w^*)
- Add vertex w^* to X and set $A[w^*] = A[v^*] + c_{v^*w^*}$

Running time?

Input: graph $G = (V, E)$ of m edges, n vertices

Loop: While $X \neq V$:

- Look at all edges (v, w) starting in X and ending in $V - X$
- Pick the one that minimizes $A[v] + c_{vw}$, call it (v^*, w^*)
- Add vertex w^* to X and set $A[w^*] = A[v^*] + c_{v^*w^*}$

- **Iterations of while loop**: $n - 1$
- **Work per iteration**: $O(m)$
- **Total complexity**: $O(mn)$

Today

Dijkstra's algorithm for shortest paths:

1. Basic version: why it works
2. **Efficient version: avoiding repeated work**

Dijkstra example, repeated

Main loop: While $X \neq V$:

- Go through all edges (v, w) starting in X, ending in $V - X$
- **Pick the edge that minimizes "Dijkstra score"** $A[v] + c_{vw}$, call it (v^*, w^*)
- Add vertex w^* to X and set $A[w^*] = A[v^*] + c_{v^*w^*}$

We are repeating calculations for some edges each time round the loop!

Efficient version: storing intermediate values

Use data structure D to store best Dijkstra scores so far

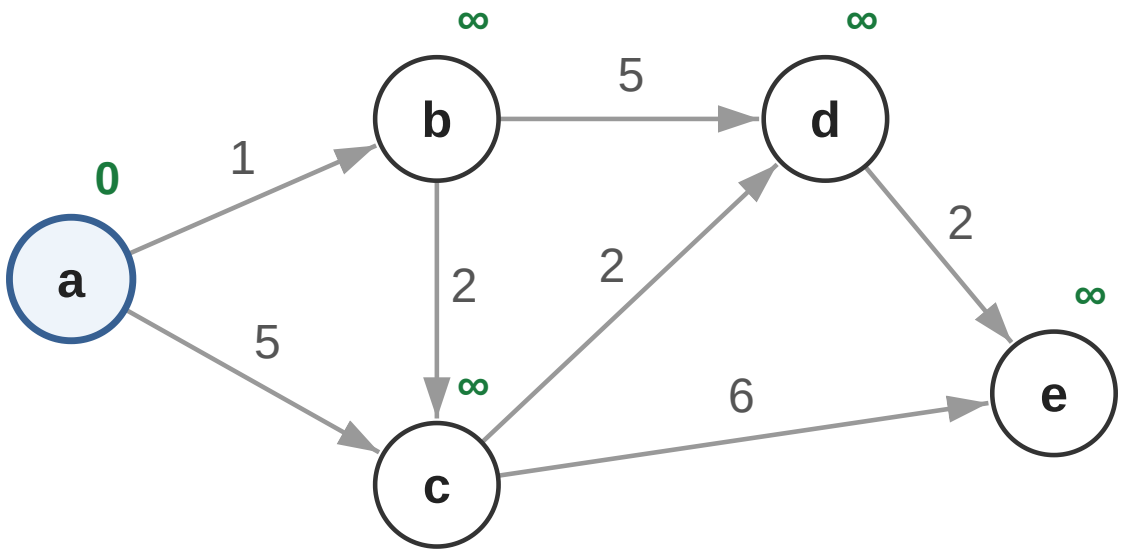
Main loop: While $X \neq V$: (not all nodes explored)

- Edges (v, w) starting from last explored node v **only**, ending in $V - X$:
  - Update Dijkstra scores in D
  - Calculate Dijkstra score of w as C, set $D(w) = \min\{D(w), C\}$
- **Remove the node with the lowest score from D, call it w^***
- Add w^* to X: it is now the last explored node

Efficient version: step 1 (initialise)

Initialise: $X = \{a\}$

Next: process edges from `a`.

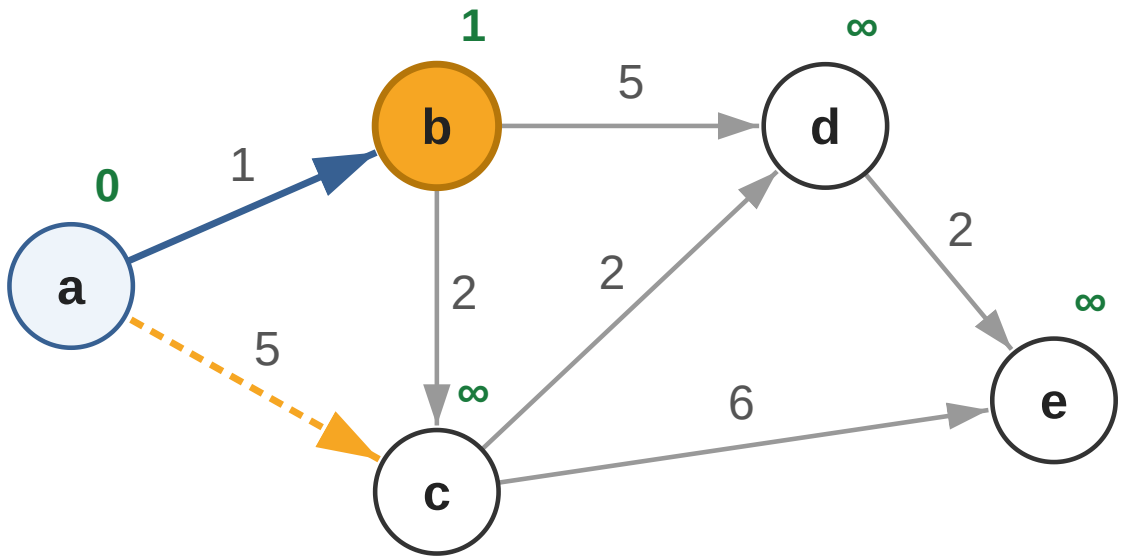



```
A = {'a': 0, 'b': inf, 'c': inf, 'd': inf, 'e': inf}
X = {'a'}
D = {'b': inf, 'c': inf, 'd': inf, 'e': inf}
```

Efficient version: step 2

Process last explored (`a`):

Edge	Score
`a` → `b`	$0 + 1 = \mathbf{1}$
`a` → `c`	$0 + 5 = 5$



Update scores: `D = {'b': 1, 'c': 5, 'd': inf, 'e': inf}`

Remove min: `b` (score 1) → add to `X`.

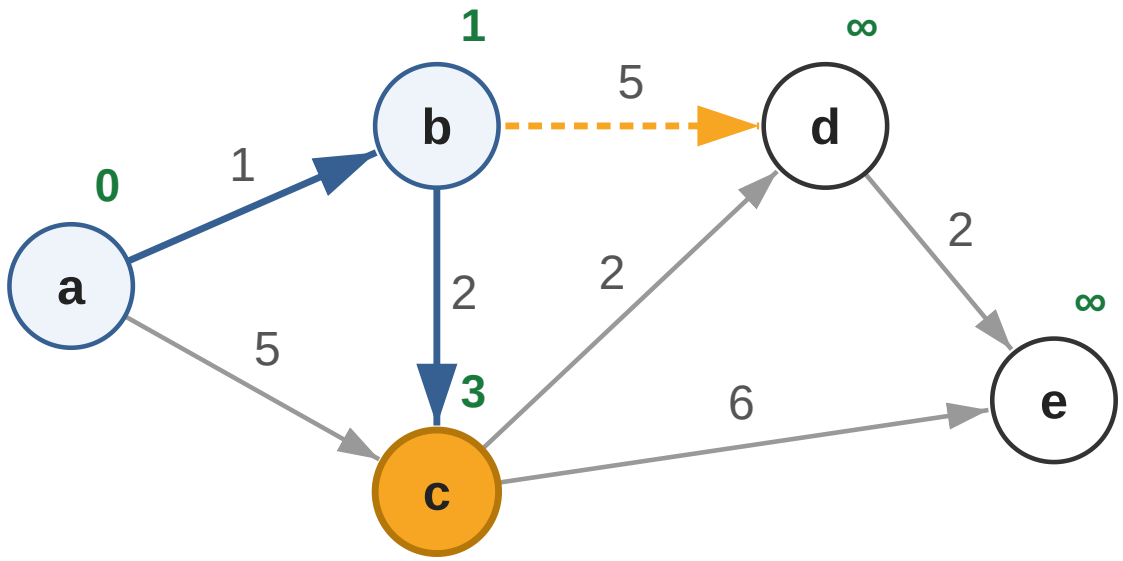
After pop: `D = {'c': 5, 'd': inf, 'e': inf}`

```
A = {'a': 0, 'b': 1, 'c': inf, 'd': inf, 'e': inf}
X = {'a', 'b'}
```

Efficient version: step 3

Process last explored (`b`) **only**:

Edge	Score
`b` → `c`	$1 + 2 = \mathbf{3}$
`b` → `d`	$1 + 5 = 6$



Update scores: `D = {'c': 3, 'd': 6, 'e': inf}`

Remove min: `c` (score 3) → add to `X`.

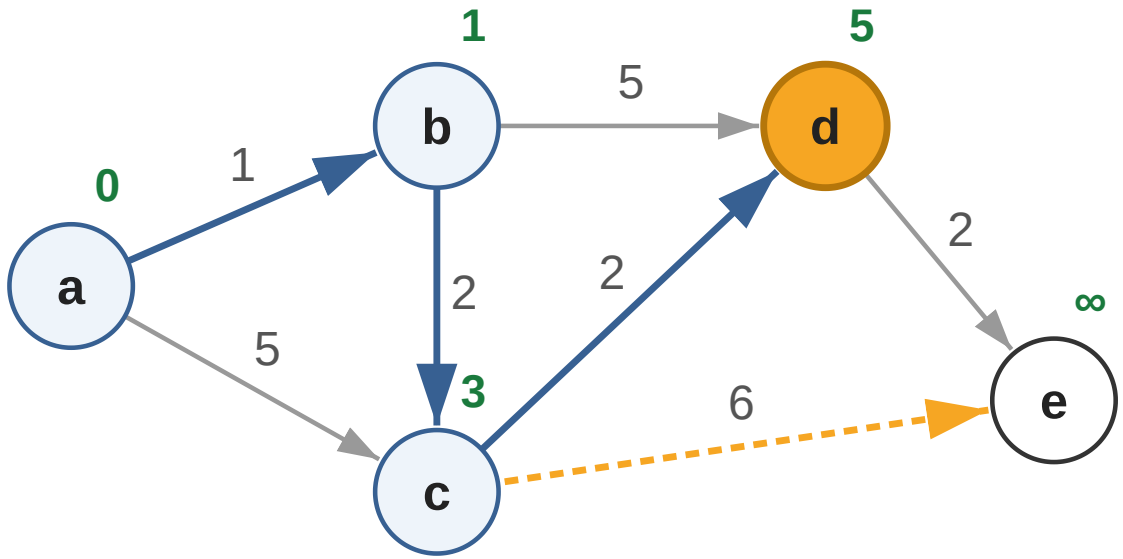
After pop: `D = {'d': 6, 'e': inf}`

```
A = {'a': 0, 'b': 1, 'c': 3, 'd': inf, 'e': inf}
X = {'a', 'b', 'c'}
```

Efficient version: step 4

Process last explored (`c`) **only**:

Edge	Score
`c` → `d`	$3 + 2 = \mathbf{5}$
`c` → `e`	$3 + 6 = 9$



Update scores: `D = {'d': 5, 'e': 9}`

Remove min: `d` (score 5) → add to `X`.

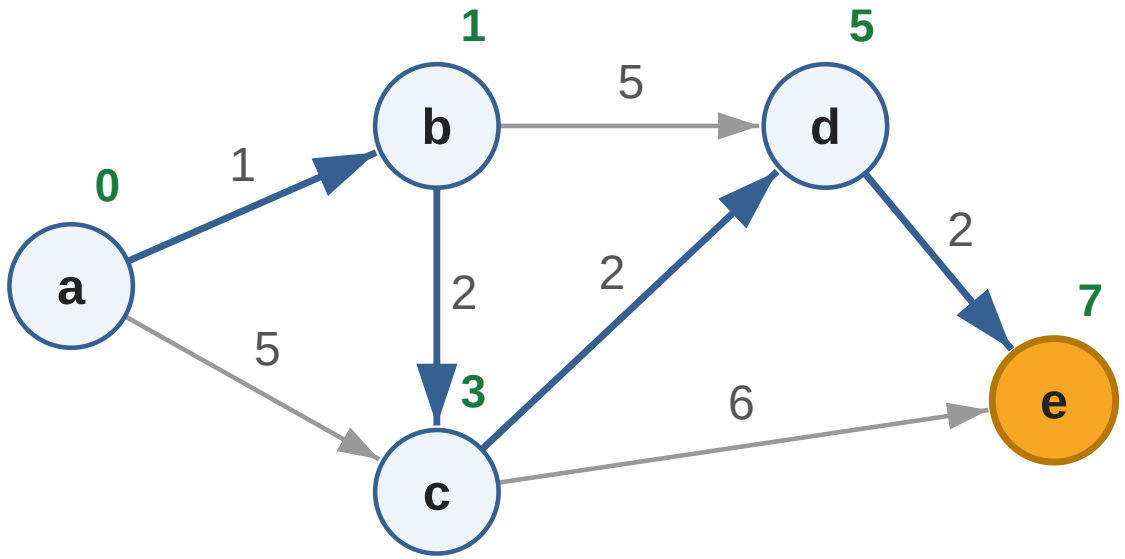
After pop: `D = {'e': 9}`

```
A = {'a': 0, 'b': 1, 'c': 3, 'd': 5, 'e': inf}
X = {'a', 'b', 'c', 'd'}
```

Efficient version: step 5

Process last explored (`d`) **only**:

Edge	Score
`d` → `e`	$5 + 2 = 7$



Update scores: `D = {'e': 7}`

Remove min: `e` (score 7) → add to `X`. All nodes explored, done!

After pop: `D = {}`

```
A = {'a': 0, 'b': 1, 'c': 3, 'd': 5, 'e': 7}
X = {'a', 'b', 'c', 'd', 'e'}
```

Why this is efficient

Notice:

- D only ever recomputes scores for edges from the **most recently explored** node
- No repeated work

Final distances: `A = {'a': 0, 'b': 1, 'c': 3, 'd': 5, 'e': 7}`

How much work is it?

Running time?

Main loop: While $X \neq V$:

- Edges (v, w) starting from last explored node v, ending in $V - X$:
  - Update Dijkstra scores in D
- **Remove the node with the lowest score from D, call it w^***
- Add w^* to X: it is now the last explored node

Need to do n times (number of nodes):

- Remove minimum from D
- Add w^* to X
- Constant work otherwise

Need to do m times (number of edges):

- Compute a Dijkstra score
- Update (or insert) score in D

Complexity of using D

Operations:

- Find minimum value in D: once per each iteration, i.e., $n - 1$ times
- Recalculate score (remove/add):
  - Once per edge: m times in total
- Total $O(m + n)$ data structure operations on D
- What happens if D is a list or a dictionary?

If D performed these operations in $O(\log n)$ time:

- Dijkstra would run in $O((m + n) \log n)$ time, instead of $O(mn)$
- Compare with BFS and queue: $O((m + n) \times 1)$

How can we do it?

Min heap:

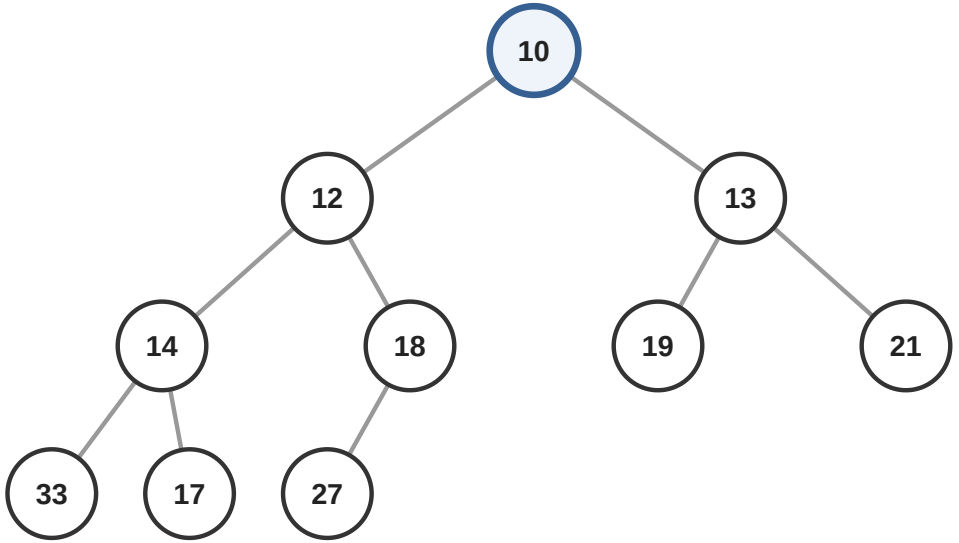
- Peek at the root (the minimum): $O(1)$
- Insert: $O(\log n)$
- Delete the root: $O(\log n)$
- Pop minimum (peek + delete): $O(\log n)$
- Essentially a **priority queue**:
  - Not FIFO: the node with the highest priority comes out first
- Our priority ordering is the Dijkstra score

How can we do it? (continued)

Implementation is a great exercise for you to try

- A problem asks you to use a built-in data structure to speed up Dijkstra
- Conceptually, a heap is a type of a **tree**:
  - The highest-priority item is at the root
  - Linked to lower-priority items in the branches
- **Difficulty**: keeping the order when adding items with different priorities

Min Heap



Definition:

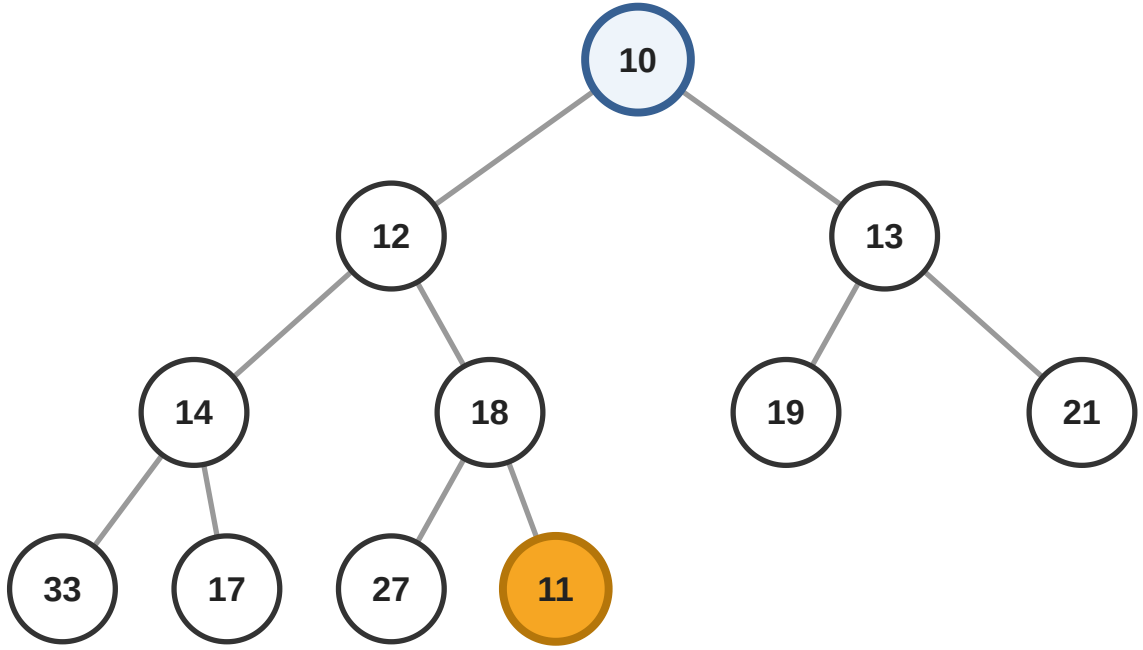
- Complete binary tree:
  - Up to two children per parent node
  - Exactly two children for all parent nodes but the last
- Min heap property: child node value $\geq$ parent node value

Min Heap (continued)

Definition (continued):

- Equivalent list: `L = [10, 12, 13, 14, 18, 19, 21, 33, 17, 27]`
- If parent is at index i, then children are at indices ...?

Add item

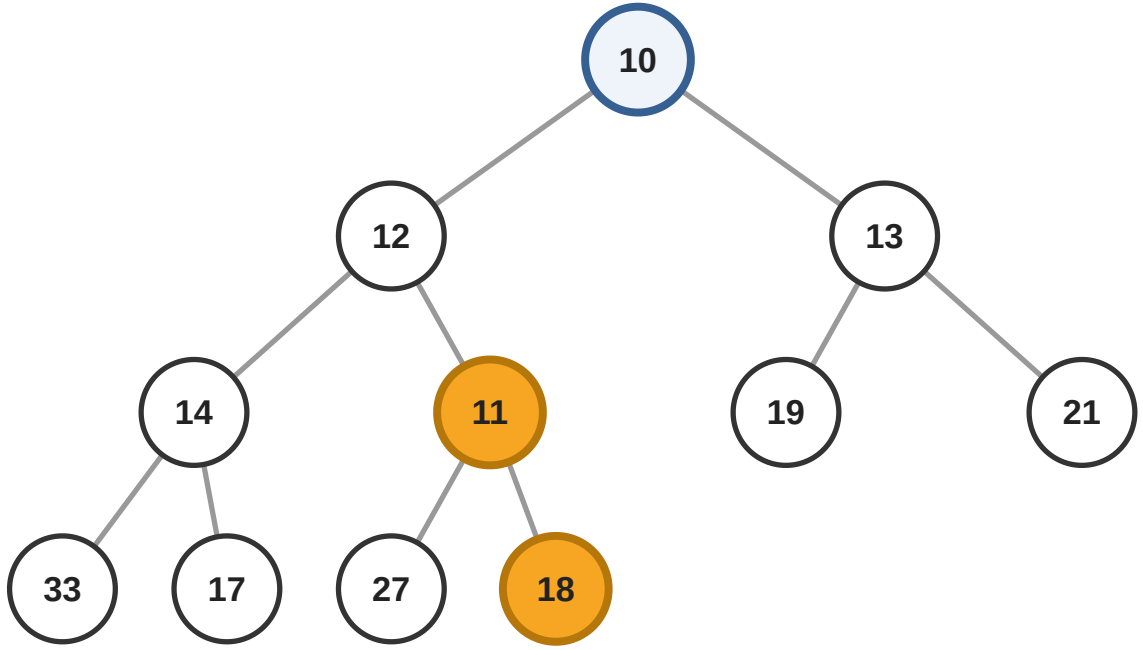


How much work is this in the worst case?

- Swap all the way to the top: how many swaps?
- Constant work per swap (using a Python list)

```
L = [10, 12, 13, 14, 18, 19, 21, 33, 17, 27, 11]
```

Add item

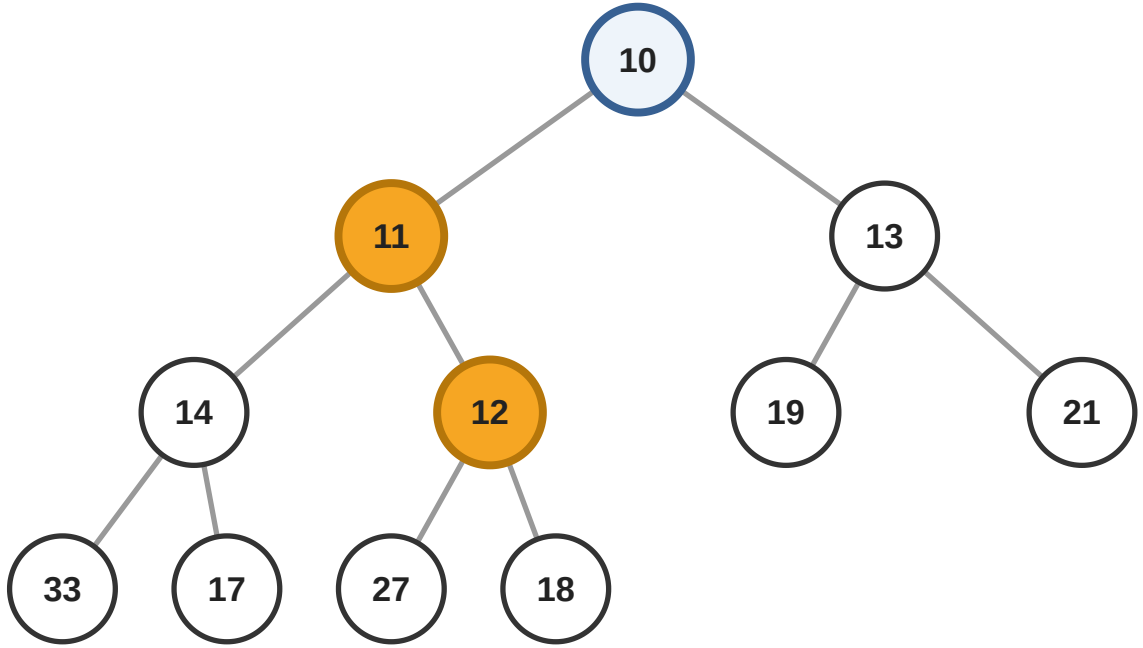


How much work is this in the worst case?

- Swap all the way to the top: how many swaps?
- Constant work per swap (using a Python list)

```
L = [10, 12, 13, 14, 11, 19, 21, 33, 17, 27, 18]
```

Add item

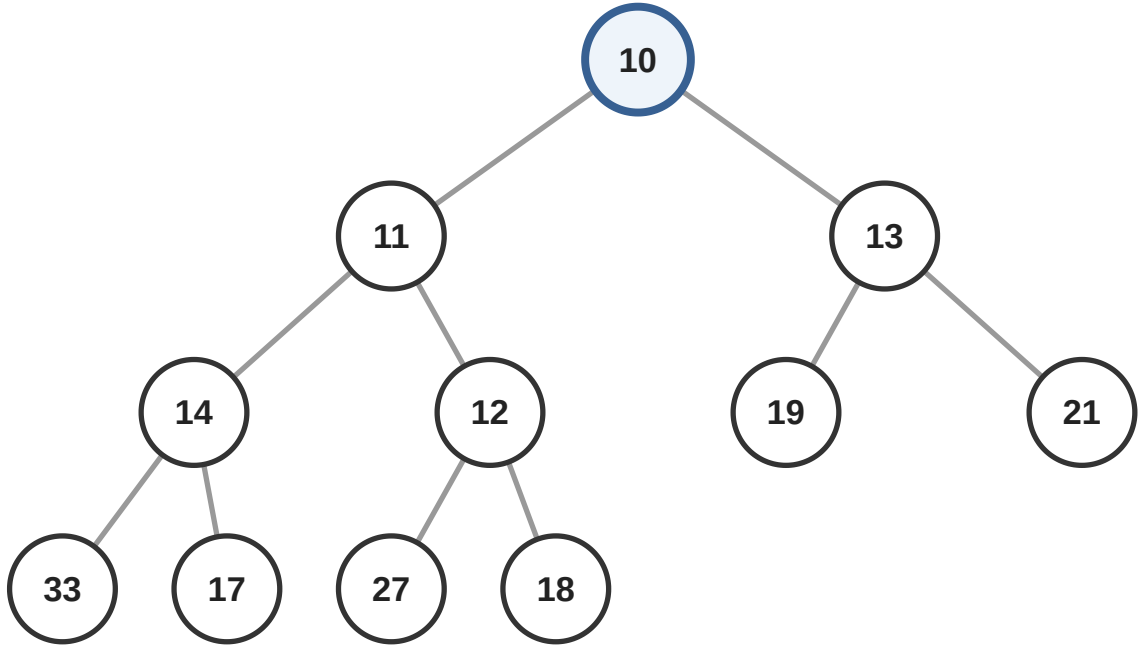


How much work is this in the worst case?

- Swap all the way to the top: how many swaps?
- Constant work per swap (using a Python list)

```
L = [10, 11, 13, 14, 12, 19, 21, 33, 17, 27, 18]
```

Pop minimum

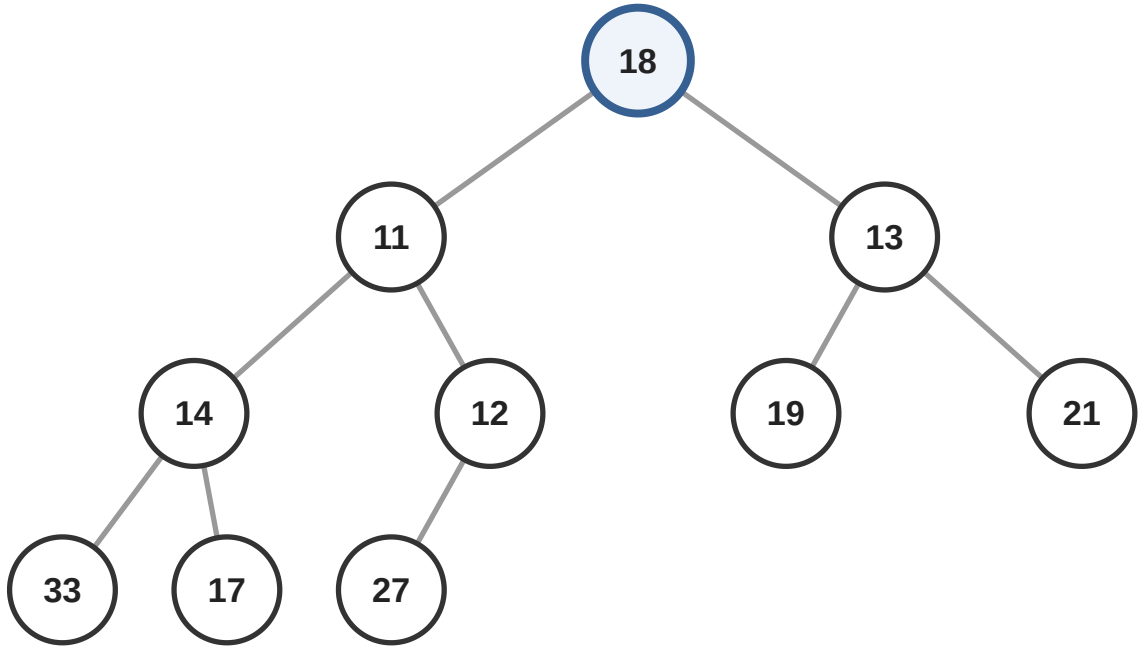


How much work is this in the worst case?

- Swap all the way to the bottom: how many swaps?
- Constant work per swap (using a Python list)

```
L = [10, 11, 13, 14, 12, 19, 21, 33, 17, 27, 18]
```

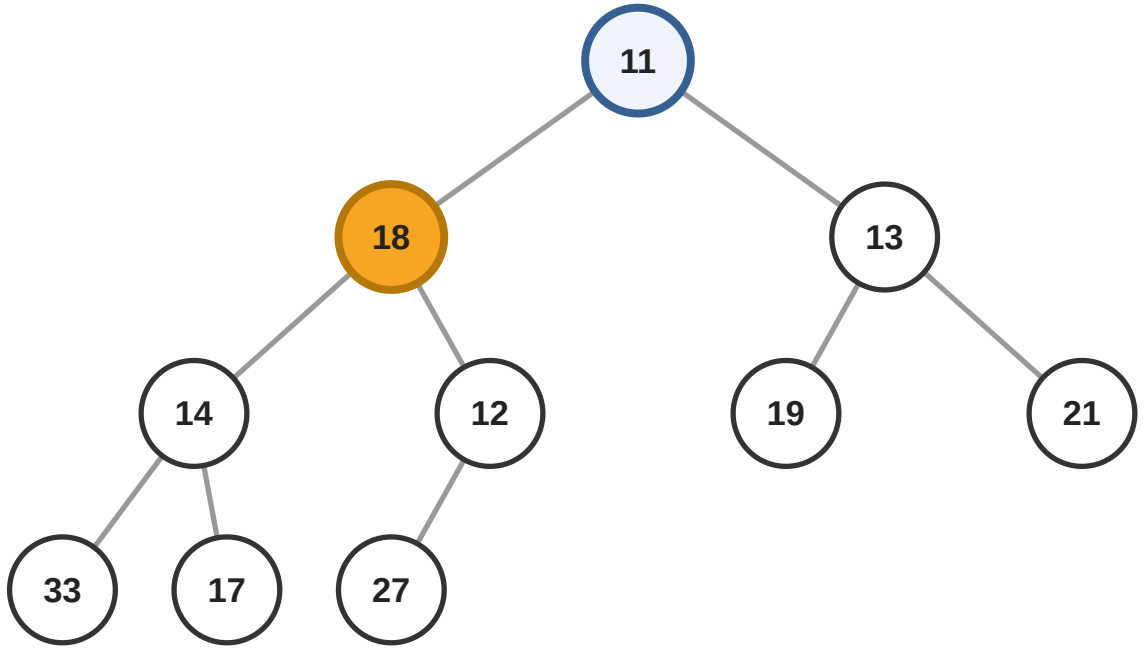
Pop minimum



Trick: swap minimum to the end and pop from list!

```
L = [18, 11, 13, 14, 12, 19, 21, 33, 17, 27]
```

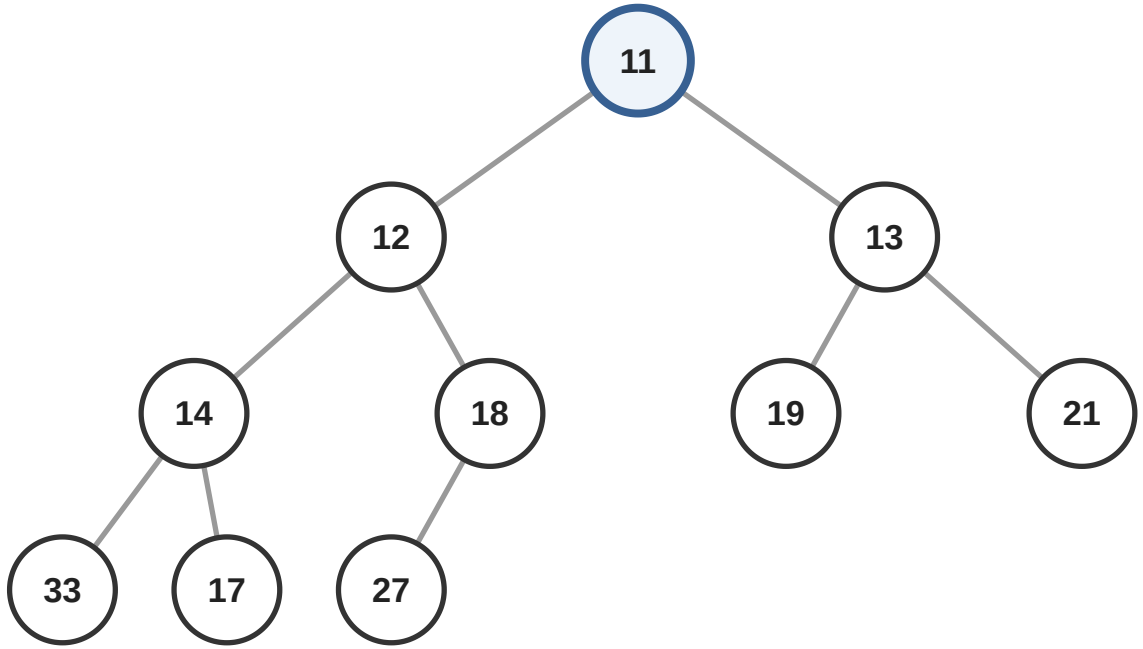
Pop minimum



Then swap all the way to the bottom: how many swaps?

```
L = [11, 18, 13, 14, 12, 19, 21, 33, 17, 27]
```

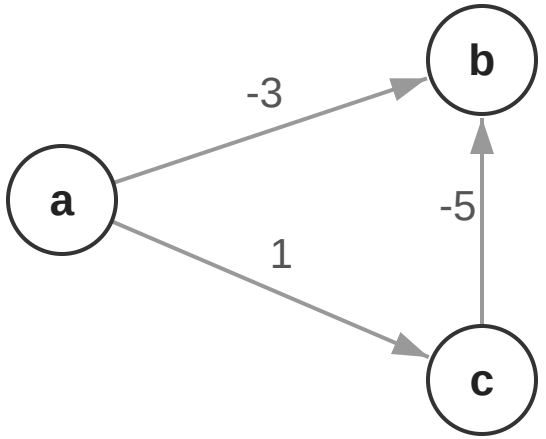
Pop minimum



Constant work per swap (using a Python list)

```
L = [11, 12, 13, 14, 18, 19, 21, 33, 17, 27]
```

We assumed non-negative edge lengths



Here Dijkstra will fail...

With negative edge weights, **we need another algorithm**:

- **Bellman-Ford**: for you to explore

Review

Shortest paths in weighted graphs

- Dijkstra's algorithm solves the problem efficiently
  - Though large problems may require more efficient algorithms
    - Specialised for the specific problem
- Data structure selection matters!
- In Dijkstra's case: heap

Exercises

- Implement Dijkstra in Python
- Recover shortest paths too...
- Travelling on the London Underground